TEKNOFEST · Yapay Zekâ Destekli Havayolu Optimizasyonu

GELİŞTİRME YOL HARİTASI

Bu çözüme ne eklenebilir?

81 somut geliştirme · formülasyonlarıyla · öncelik matrisi ve faz planı

Takım Büke

Bu doküman ne işe yarar? İki amacı var. Birincisi: jüri neredeyse kesin olarak «bundan sonra ne yapardınız?» diye soracak — bu soruya «gap'i kapatırdık» gibi tek cümlelik bir cevap vermek zayıf kalır. Burada 81 somut madde, çoğu matematiksel formülasyonuyla birlikte hazır. İkincisi: projenin gerçekten devam ettirilmesi hâlinde izlenecek yol haritası. Her madde için ne / neden / nasıl / tahmini etki / zorluk / gereken veri verilmiştir.

İçindekiler

0. **Nasıl kullanılır ve öncelik özeti** *(—)*

1. **Hızlı kazanımlar — günler içinde yapılabilir** *(14 madde)*

2. **Model genişletmeleri — yeni kısıt ve amaç terimleri** *(12 madde)*

3. **Çözüm yöntemi iyileştirmeleri** *(10 madde)*

4. **Belirsizlik ve sağlamlık (stokastik / robust)** *(6 madde)*

5. **Rekabet dinamikleri — rakip artık sabit değil** *(4 madde)*

6. **Talep ve gelir modellemesi** *(6 madde)*

7. **Yapay zekâ entegrasyonu** *(9 madde)*

8. **Yazılım mimarisi ve ürünleşme** *(8 madde)*

9. **Test, doğrulama ve kalite güvencesi** *(6 madde)*

10. **Görselleştirme ve karar destek arayüzü** *(6 madde)*

11. **Yol haritası — öncelik matrisi ve faz planı** *(—)*

12. **Sunumda hangileri söylenmeli?** *(—)*

0. Nasıl kullanılır ve öncelik özeti

0.1 Maddelerin okunuşu

Her madde şu alanları taşır:

Alan	Anlamı
Ne	Somut olarak eklenecek şey
Neden	Hangi eksiği / zayıflığı kapatıyor
Nasıl	Teknik yöntem — çoğu maddede matematiksel formülasyonuyla
Etki	Beklenen kazanç (mümkün olduğunda ölçülmüş rakamla)
Zorluk	Çok düşük / Düşük / Orta / Yüksek / Araştırma konusu
Gereken veri	Elimizde olmayan ve tedarik edilmesi gereken veri

0.2 En yüksek öncelikli 10 madde

Sınırlı zamanınız varsa şu sırayla ilerleyin. İlk dördü ölçülmüş açıkları kapatır, yani en yüksek getiri/maliyet oranına sahiptir.

#	Madde	Neden öncelikli	Zorluk
1	1.1 Rotasyon filtresini çift bazlı yapmak	221 uçuşta A2 kısıtı kurulamıyor; kaydırmaların %23,2'si korumasız	Çok düşük
2	1.2 Baseline'ı ızgara-tutarlı raporlamak	Kazancın %14,1'i (+21.597 puan) ayrıklaştırmadan geliyor	Çok düşük
3	2.1 Soft-E2 (ihlali yasaklamak yerine cezalandırmak)	Kapsam kriterinin (%30) en büyük açığını kapatır	Orta
4	2.2 Bağlantı ödülünü bekleme süresine göre ağırlıklandırmak	«Yeni bağlantılarınız pencerenin en kötü ucunda» itirazını kökten çözer	Düşük
5	3.1 Ticari çözücü (indicator constraints)	E2'yi big-M'siz yazma + gap'i daraltma	Düşük (lisans meselesi)
6	2.4 Uçak kuyruk (tail) sürekliliği	Operasyonel uygulanabilirliğin en kritik eksiği	Orta
7	9.1 Birim testler	Şu an hiç otomatik test yok; regresyon riski yüksek	Düşük
8	7.1 ML ile bağlantı çekiciliği tahmini	Yarışmanın «yapay zekâ destekli» başlığına doğrudan cevap	Orta
9	2.3 Pazar bazlı koruma kısıtı	24 pazar-günde sıralama kötüleşiyor; yönetimsel kabul için gerekli	Düşük
10	3.4 Daha uzun cilalama + LNS	%1,18 gap'i kapatır (~40.000 puanlık potansiyel)	Düşük

0.3 Etki × maliyet matrisi (kaba yerleşim)

	DÜŞÜK maliyet	**ORTA maliyet**	**YÜKSEK maliyet**
YÜKSEK etki	1.1 rotasyon filtresi · 1.2 ızgara-tutarlı baseline · 2.2 kalite ağırlığı · 3.1 ticari çözücü · 3.4 uzun cilalama	2.1 soft-E2 · 2.4 tail sürekliliği · 7.1 ML çekicilik · 6.2 logit pazar payı	4.2 stokastik gecikme · 5.1 oyun teorik denge · 2.6 ekip kısıtları
ORTA etki	1.3–1.14 diğer hızlı kazanımlar · 2.3 pazar koruma · 9.1 birim testler	3.2 Benders · 3.3 column generation · 7.3 ML-guided branching · 10.x görselleştirme	8.5 tam ürünleşme · 4.4 senaryo bazlı stokastik
DÜŞÜK etki	1.7 sürüm sabitleme · 1.9 ölü kod temizliği	3.9 simetri kırma · 9.4 property-based test	—

1. Hızlı kazanımlar — günler içinde yapılabilir

Bu bölümdeki maddeler mevcut mimariye dokunmadan, çoğu birkaç satırlık değişiklikle uygulanabilir. Dördü ön-denetimde ölçülmüş gerçek açıkları kapatır.

1.1 Rotasyon eşleştirme filtresini ÇİFT bazlı yapmak

Ne: Flight Pairs filtresi uçuş bazlı; çift bazlı olmalı.

Neden: Ölçülen açık: 512 Pair'in yalnız 291'i kullanılabilir (gidiş, dönüş) çifti üretiyor; 221 uçuşta A2 rotasyon kısıtı hiç kurulamıyor ve bunların 100'ü çözümde oynatılmış (248 hareket = tüm kaydırmaların %23,2'si).

Nasıl: Mevcut filtre `rot = pair[pair.FlNo.isin(tk_flnos)]` uçuş bazlıdır. Bir Pair grubunun her iki bacağı da veri setinde yoksa çift yarım kalır. İki seçenek var: (a) yarım kalan çiftlerdeki uçuşları da `tk_flights_without_partner` listesine ekleyip SABİT kabul etmek — güvenli, kazancı bir miktar düşürür; (b) çifti tam tutup eksik bacağı gerçek saatiyle ANKORLAMAK — U9'da yapılanın aynısı, kazancı korur.

```
# MEVCUT (ucus bazli — 221 ucus korumasiz kaliyor)
rot = pair[pair.FlNo.isin(tk_flnos)]

# ONERI (a): cift bazli filtre + yarim kalanlari sabitle
ok_pairs = [p for p, g in pair.groupby('Pair')
            if (g.Orig == 'IST').any() and (g.Dest == 'IST').any()
            and g.FlNo.isin(tk_flnos).all()]
rot      = pair[pair.Pair.isin(ok_pairs)]
half_fl  = set(pair[~pair.Pair.isin(ok_pairs)].FlNo) & set(tk_flnos)
nopart   = sorted(set(nopart) | half_fl)     # -> bu ucuslar SABIT kabul edilir
```

Etki: A2 kapsamı %57'den ~%100'e; fiziksel garanti tamamlanır · **Zorluk:** Çok düşük (yarım gün) · **Gereken veri:** Yok — mevcut veriyle yapılabilir

1.2 Baseline'ı ızgara-tutarlı raporlamak (çift raporlama)

Ne: Baseline'ı hem ham dakikayla hem 10 dk ızgarasıyla hesaplayıp ikisini de raporlamak.

Neden: Ölçülen açık: baseline `gap_cur = gap_min` (ham dakika) kullanırken model 10 dk ızgaraya yuvarlıyor. 7.577 adayın 4.395'inde iki değer farklı; 89 aday pencere dışından içine kayıyor. Etki: +21.597 puan = kazancın %14,1'i. Izgara-tutarlı iyileşme +%4,72 yerine +%4,03.

Nasıl: `current_baseline()` fonksiyonuna bir bayrak ekleyip iki değeri de hesaplamak ve rapora İKİ SATIR yazmak. Model değişmiyor, sadece raporlama dürüstleşiyor.

```
def current_baseline(inst, grid_consistent=False):
    ...
    if grid_consistent:
        g = snap(dep_cur) - snap(arr_cur)      # modelin gordugu bosluk
    else:
        g = conns[pi]['gap_cur']               # HAM dakika (mevcut davranis)
    offered = [pi for pi in ... if L <= g <= U]

# raporda:
#   BASELINE (ham)            : 3.251.527   -> iyilesme +153.512 (+%4,72)
#   BASELINE (izgara-tutarli) : 3.273.124   -> iyilesme +131.915 (+%4,03)
```

Etki: Metodolojik dürüstlük; jüri itirazını önceden karşılar · **Zorluk:** Çok düşük (1–2 saat)

1.3 Kök düğüm LP bound'unu raporlamak

Ne: Çözücüden kök düğümün LP relaxation değerini alıp raporlamak.

Neden: «Sıkı doğrusallaştırma» iddiamızı destekleyen tek bir sayı bile yok. Bir yöneylem hakemi «integrality gap kaç?» diye sorarsa cevabımız yok.

Nasıl: HiGHS çözüm sonrası `getInfo()` ile kök bound alınabilir; ya da `mip_rel_gap=1e9` ile bir kez çalıştırıp ilk bound'u okumak. Rapora «kök LP: X, incumbent: Y, integrality gap: %Z» satırı eklenir.

Etki: Formülasyon kalitesi iddiasını sayısallaştırır · **Zorluk:** Çok düşük

1.4 ρ çakışmalarını açık politikayla çözmek

Ne: Yolcu verisinde çelişkili ρ değeri olan 6 pazar için belgeli bir kural.

Neden: PEK–AGP 45 mi 627 mi (13,9 kat fark), PEK–VCE 73 mü 548 mi… Şu an pandas varsayılanıyla ilk satır alınıyor — yani seçim Excel satır sırasına bağlı ve hiçbir yerde belgelenmemiş.

Nasıl: CFG'ye `rho_conflict` politikası eklemek: 'first' | 'max' | 'mean' | 'error'. Varsayılan 'error' olup açık uyarı vermesi en güvenlisidir; çünkü sessiz seçim gizli test verisinde fark edilmez.

```
dup = pax[pax.market.duplicated(keep=False)]
conf = dup.groupby('market').rho.nunique()
conf = conf[conf > 1]
if len(conf):
    log(f'[UYARI] {len(conf)} pazarda CELISKILI rho: {list(conf.index)}')
    if CFG['rho_conflict'] == 'error':
        raise ValueError('Celiskili rho — politika belirleyin')
    pax = pax.groupby('market', as_index=False).rho.agg(CFG['rho_conflict'])
```

Etki: Sessiz veri seçimini ortadan kaldırır · **Zorluk:** Çok düşük

1.5 MCT güvenlik payı seçeneği (L = 75 dk)

Ne: Bağlantı alt sınırını parametrik olarak yükseltme imkânı.

Neden: Ölçüm: tam 60 dakikada bekleyen bağlantı sayısı baseline'da 202 (%3,0) iken bizde 541 (%7,8); ≤70 dk olanlar 623 → 944. Model MCT sınırına doğru itiyor ve 15 dakikalık bir gecikme bu bağlantıları öldürür.

Nasıl: L zaten CFG'de. Yapılacak tek şey L=75 ile bir koşu alıp kazanç kaybını ölçmek ve «sağlamlık ↔ kazanç» ödünleşimini bir tabloyla raporlamak.

Etki: Operasyonel sağlamlık; ölçülebilir ödünleşim tablosu · **Zorluk:** Çok düşük

1.6 İki çıktı arasındaki «yenilen rakip» tutarsızlığını gidermek

Ne: market_results'taki β tabanlı sayı ile selected_connections'tan yeniden hesaplanan sayıyı hizalamak.

Neden: β yalnız üst sınırla (β ≤ Σw) tanımlı olduğundan, çözüm %0,1 gap'te durdurulduğunda bazı β'lar 1'e çıkmamış olabilir. Doğrulayıcı bunu 1.199 pazar-günde «ihtiyatlı eksik-beyan» olarak raporluyor. Amacı şişirmiyor ama iki çıktı dosyası arasında görünür tutarsızlık yaratıyor.

Nasıl: Çözüm sonrası bir «temizleme» adımı: her pazar-gün için gerçek yenilen sayısını seçilen bağlantılardan yeniden hesaplayıp market_results'a yazmak. Amaç değeri değişmez (raporlanan zaten β'dan okunuyor), sadece raporlama tutarlı olur.

Etki: Çıktılar arası tutarlılık · **Zorluk:** Çok düşük

1.7 requirements.txt'te sürümleri sabitlemek

Ne: `pyomo>=6.7` yerine `pyomo==6.10.1` gibi tam sürüm.

Neden: Farklı bir HiGHS sürümü farklı bir çözüm bulabilir; tekrar-üretilebilirlik iddiasının teknik açığı.

Nasıl: `pip freeze` çıktısından üretilen bir `requirements.lock` dosyası eklemek; `requirements.txt` esnek kalabilir.

Etki: Tekrar-üretilebilirlik · **Zorluk:** Çok düşük

1.8 rank_reward'da eksik lookup anahtarına uyarı

Ne: Lookup'ta bulunamayan (N, b, s) üçlüsünde sessizce 0 dönmek yerine uyarmak.

Neden: Mevcut veride N ≤ 17 ve lookup N ≤ 40 olduğu için sorun yok. Ama gizli test verisinde N > 40 çıkarsa ilgili pazarların TÜM sıralama ödülü sessizce sıfırlanır.

Nasıl: Fonksiyona bir sayaç ekleyip koşu sonunda «X pazar-günde lookup anahtarı bulunamadı» uyarısı basmak.

Etki: Gizli veri güvenliği · **Zorluk:** Çok düşük

1.9 Ölü kodun temizlenmesi

Ne: `set_warmstart()` (03'te, ~52 satır) ve `tmin()` (02'de) fiilen kullanılmıyor.

Neden: 03'te `CFG['warmstart'] = False` olduğu için `set_warmstart` hiç çağrılmıyor; ama modelle senkron kalması gereken ikinci bir «çözüm mantığı» kopyası olarak duruyor. İleride model değişirse sessizce sapabilir.

Nasıl: Silmek veya 08'in `set_start_values()` fonksiyonuyla birleştirmek. İkisi zaten aynı işi yapıyor.

Etki: Bakım yükü azalır, sapma riski biter · **Zorluk:** Çok düşük

1.10 Hesaplanan ama raporlanmayan sayaçları basmak

Ne: `m._n_f_overflow`, `n_band_dropped`, `m._n_rot_chain` hesaplanıyor ama hiçbir yere yazılmıyor.

Neden: Bunlar tam olarak jürinin sorabileceği türden şeffaflık göstergeleri — «kapasitede kaç taşma oldu?», «kaç bant bırakıldı?»

Nasıl: Koşu özetine ve `results_summary.md`'ye eklemek.

Etki: Şeffaflık · **Zorluk:** Çok düşük

1.11 Gün sırasının etkisini ölçmek

Ne: Güne göre ayrıştırmada G bandı İLK çözülen günde çapalanıyor; farklı gün sırası farklı sonuç verir.

Neden: `band.setdefault(...)` ile merkez ilk günde kilitleniyor. Gün 1 hiçbir bant kısıtı olmadan çözülüyor, sonrakiler ona zorlanıyor. Bu bir greedy tercihtir ve sonucu etkiler ama hiç ölçülmemiş.

Nasıl: 7 farklı gün permütasyonuyla (veya rastgele 10 sırayla) koşup sonuç dağılımını raporlamak. Ayrıca «en kısıtlı günden başla» gibi bir sıralama sezgiseli denenebilir.

Etki: Ayrıştırmanın duyarlılığı belgelenir; muhtemelen küçük ek kazanç · **Zorluk:** Düşük

1.12 Gece yarısı sarmalama (wrap) kenar durumunu sağlamlaştırmak

Ne: Saat-içi aritmetiği modüler değil; gece yarısını geçen kaydırmalarda G bandı karşılaştırması sarmalamayı hesaba katmıyor.

Neden: Ofset her yerde `⌊cur/1440⌋·1440` ile MEVCUT saatten alınıyor. Optimize saat gece yarısını geçerse θ' değeri [0,1440) dışına çıkabilir. Mevcut veride sorun yaratmıyor ama gizli veride yaratabilir.

Nasıl: Karşılaştırmaları modüler yapmak: `d = min(|a−b|, 1440−|a−b|)`. Doğrulayıcıda (05) da aynı düzeltme yapılmalı ki iki taraf tutarlı kalsın.

Etki: Kenar durum sağlamlığı · **Zorluk:** Düşük

1.13 Kısıtları aileye göre isimlendirmek

Ne: Tüm kısıtlar tek bir `ConstraintList` içinde; hangi kısıtın hangi aileye ait olduğu izlenemiyor.

Neden: Model infeasible olduğunda hangi kısıt ailesinin suçlu olduğunu bulmak zor. IIS (irreducible infeasible subsystem) analizi de yapılamıyor.

Nasıl: Her aile için ayrı `ConstraintList` (m.cons_A1, m.cons_B1, …) veya isimli `Constraint` blokları kullanmak. Çözücü çıktısında ve hata ayıklamada aile bazlı izlenebilirlik sağlar.

Etki: Hata ayıklama ve IIS analizi mümkün olur · **Zorluk:** Düşük

1.14 Duyarlılık analizini genişletmek

Ne: Şu an yalnız 3 parametre taranıyor (λ, Wdev, τ); CFG'de ~15 ayarlanabilir parametre var.

Neden: Taranmayanlar: cap_dep/cap_arr (hub kapasitesi), alpha (E1 toleransı), xdev_min (G bandı), L/U (bağlantı penceresi), delta_min (ızgara çözünürlüğü). Ayrıca tasarım tek-faktörlü (OAT) olduğu için etkileşimleri göremiyor.

Nasıl: En az 6 parametreye çıkarmak ve λ×Wdev gibi 2 faktörlü tam faktöriyel bir alt tarama eklemek. Ayrıca Wdev=±90 ölçümünü daha uzun süre limitiyle tekrarlamak (mevcut ölçüm zaman-limitli ve bilimsel olarak geçersiz).

Etki: Modelin parametre uzayındaki davranışı belgelenir · **Zorluk:** Düşük

2. Model genişletmeleri — yeni kısıt ve amaç terimleri

Bu bölümdeki maddeler modele yeni matematiksel yapı ekler. Her biri için formülasyon verilmiştir; hepsi DOĞRUSALDIR, yani mevcut MIP yapısını bozmaz.

2.1 Soft-E2 — yönsel süre dengesini ceza olarak modele almak

Ne: E2'yi sert kısıt yerine amaçta cezalandırılan bir hedef hâline getirmek.

Neden: Şu an E2 tam ölçekte kapalı; bu, model kapsamı kriterinin (%30) en büyük açığı. Sert uygulanamamasının sebebi hem çözücü (big-M LP relaxation'ı bozuyor) hem veri (aktif yön çiftlerinin %76'sı eşiği 465 dakikaya varan farklarla ihlal ediyor). Soft yapıldığında model onu YASAKLAMAZ, MİNİMİZE EDER.

Nasıl: İhlal miktarı için sürekli bir değişken tanımlanır ve amaca ρ-ağırlıklı bir ceza terimi eklenir. Yasak olmadığı için infeasibility riski yoktur.

$$e_{od,h} \geq Jmin_{od,h} - Jmin_{do,h} - \Gamma \qquad \text{(ihlal miktarı, tek yön)}$$
$$e_{od,h} \geq Jmin_{do,h} - Jmin_{od,h} - \Gamma \qquad \text{(diğer yön)}$$
$$e_{od,h} \geq 0$$

$$\text{AMAÇ} = \Sigma\, \rho\cdot(\text{bağlantı} + \text{sıralama}) - \lambda\cdot\Sigma|\theta - \text{cur}| - \mu \cdot \Sigma\, \rho_{od} \cdot e_{od,h}$$

```
m.e = pyo.Var(md_index, domain=pyo.NonNegativeReals)
for k in md_pairs:                      # (od, do) cifti, her ikisi de aktifse
    i, j = md_id[k[0]], md_id[k[1]]
    m.cons.add(m.e[i] >= m.Jmin[i] - m.Jmin[j] - GAMMA)
    m.cons.add(m.e[i] >= m.Jmin[j] - m.Jmin[i] - GAMMA)
obj -= MU * sum(rho[k] * m.e[md_id[k]] for k in md_pairs)
```

Etki: E2 modele girer; ihlal ortalaması ölçülebilir hâle gelir. μ için λ'daki gibi bir Pareto taraması gerekir. · **Zorluk:** Orta — Jmin değişkenleri hâlâ big-M gerektiriyor; §3.1 ile birlikte yapılırsa çok daha kolay

2.2 Bağlantı ödülünü bekleme süresine göre ağırlıklandırmak

Ne: Her bağlantıya, bekleme süresine bağlı bir kalite çarpanı eklemek.

Neden: Ölçülen sorun: model 60 dakikalık bir bağlantıyla 300 dakikalık bağlantıyı AYNI sayıyor. Sonuç: pencereye yeni giren 529 bağlantının %94'ü 270–300 dakika bandında. Genel ürün karması yine de iyileşiyor (60–90 dk bandı +145) ama metrik kaliteyi görmüyor.

Nasıl: Ödülü parçalı doğrusal (piecewise linear) bir kalite fonksiyonuyla çarpmak. x[olay,saat] yapısı sayesinde bu DOĞRUSAL kalır: her aday saat kombinasyonunun beklemesi önceden bilindiği için kalite çarpanı bir SABİTTİR.

$$q(\text{gap}) = \begin{cases} 1{,}00 & \text{eğer } 60 \leq \text{gap} \leq 120 \quad \text{(çok iyi ürün)} \\ 0{,}85 & \text{eğer } 120 < \text{gap} \leq 180 \\ 0{,}65 & \text{eğer } 180 < \text{gap} \leq 240 \\ 0{,}45 & \text{eğer } 240 < \text{gap} \leq 300 \quad \text{(zayıf ürün)} \end{cases}$$

$$\text{bağlantı ödülü} = \Sigma_j\ W^c_j \cdot q_\pi \cdot z_{od,h,j}$$

Etki: Ürün kalitesi eleştirisini kökten çözer; muhtemelen 240–300 bandındaki kazanımı azaltıp 60–120 bandını artırır · **Zorluk:** Düşük — q sabit olduğu için doğrusallık korunur · **Gereken veri:** Yok; ağırlıklar THY gelir yönetimiyle kalibre edilmeli

2.3 Pazar bazlı koruma kısıtı

Ne: «Hiçbir pazarda sıralama k basamaktan fazla düşmesin» kuralı.

Neden: Çözüm 24 pazar-günde sıralamayı kötüleştiriyor (1–6 basamak). Toplam faydada telafi ediliyor ama bir havayolu için «hangi pazarda geri gitmeye razıyız» bir yönetim kararıdır, optimizasyon kararı değil.

Nasıl: γ değişkenleri zaten son sırayı seçiyor. Basit bir doğrusal kısıt yeterli: seçilen son sıra, taban sıradan en fazla k fazla olabilir.

$$\Sigma_b\ b \cdot \gamma_{od,h,b} \geq N_{od,h} - b_{od} - k \qquad \forall\, (o,d,h) \in \mathcal{S}^{\text{protekt}}$$

$$k = 0 \rightarrow \text{hiçbir pazarda gerileme yok (en katı)}$$
$$\mathcal{S}^{\text{protekt}} = \text{korunacak stratejik pazarlar listesi (ticari karar)}$$

Etki: Yönetimsel kabul edilebilirlik; kazançta küçük bir düşüş beklenir · **Zorluk:** Düşük

2.4 Uçak kuyruk (tail) sürekliliği

Ne: Bir uçağın gün boyu yaptığı TÜM turların zincirini modellemek.

Neden: Şu an yalnız ikili gidiş-dönüş turu korunuyor. Gerçekte bir uçak günde 3–4 tur yapar: IST→A→IST→B→IST. B turunu öne çekerseniz uçak A'dan dönmemiş olabilir. Bu, operasyonel uygulanabilirliğin en kritik eksiği.

Nasıl: İki yaklaşım var. (a) Veri varsa: her kuyruk için ardışık görevleri sıralayıp her ardışık çifte bir minimum bağlantı kısıtı koymak. (b) Veri yoksa: uçuşları kuyruğa ATAYAN ikili değişkenler eklemek — bu problemi «fleet/tail assignment» ile birleştirir ve modeli belirgin büyütür.

$$\text{Kuyruk t'nin görev dizisi } r_1 \to r_2 \to \ldots \to r_k \text{ için:}$$

$$\theta_\{\text{varış}\}(r_i) \;+\; \tau_\text{ground} \;\le\; \theta_\{\text{kalkış}\}(r_{i+1}) \qquad \forall\, i$$

Etki: Tarifenin gerçekten uçurulabilir olduğunu garanti eder · **Zorluk:** Orta (veri varsa) / Yüksek (atama da yapılacaksa) · **Gereken veri:** Kuyruk numarası bazlı görev dizisi — mevcut veride YOK

2.5 Dış istasyon slot ve curfew kısıtları (Kısıt H)

Ne: Slot-koordineli havalimanlarında varış/kalkış pencerelerini daraltmak.

Neden: Bir uçuşun IST kalkışını 40 dakika öne çekerseniz varış havalimanındaki slotu da kayar. Ölçüm: kaydırılan hareketlerin 250'si (%23,4) 00:00–06:00 bandında, 211'i (%19,8) uzun menzil. Bu gruplar slot ve gece yasağına en duyarlı olanlar.

Nasıl: Mimarî olarak trivial: etkilenen olayların aday saat kümesi $\mathcal{T}_r$ daraltılır. Model yapısı hiç değişmez, sadece daha az aday üretilir — üstelik model KÜÇÜLÜR.

```
# get_event icinde, aday uretiminden sonra:
if (istasyon, rol) in slot_pencereleri:
    lo, hi = slot_pencereleri[(istasyon, rol)]
    varis_saatleri = [v + blok for v in cands]         # varis istasyonundaki saat
    cands = [v for v, a in zip(cands, varis_saatleri) if lo <= a % 1440 <= hi]
```

Etki: Operasyonel geçerlilik; model boyutu küçülür · **Zorluk:** Çok düşük (kod) — veri tedariki asıl iş · **Gereken veri:** Havalimanı bazlı slot pencereleri ve curfew saatleri

2.6 Ekip (crew) uyumluluk kısıtları

Ne: Kaydırmaların ekip görev süresi (FDP) ve dinlenme kurallarını bozmamasını sağlamak.

Neden: Bir uçuşu 40 dakika kaydırmak kokpit/kabin görev süresini, dinlenme sürelerini ve layover otel rezervasyonlarını etkiler. Bazı pairing'ler yasal limite takılır.

Nasıl: Tam bir crew pairing modeli çok büyük olur. Pratik yaklaşım: mevcut pairing'leri veri olarak alıp, her pairing için toplam görev süresinin limiti aşmamasını sağlayan doğrusal bir kısıt eklemek.

$$\text{Pairing p'deki ilk görevin başlangıcı ile son görevin bitişi arasında:}$$

$$\theta_\text{son_varış}(p) \;-\; \theta_\text{ilk_kalkış}(p) \;\le\; \text{FDP_max}(p)$$

Etki: Uygulanabilirlik; kazançta düşüş beklenir · **Zorluk:** Yüksek · **Gereken veri:** Ekip pairing yapısı ve FDP limitleri

2.7 Bakım pencereleri

Ne: Uçakların planlı bakım için belirli saatlerde belirli üslerde olmasını zorlamak.

Neden: Gece bakımı genelde hub'da veya belirli bir üste yapılır. Bir uçağın son turunun IST'e dönüş saati bakım penceresini kaçırırsa bakım kayar.

Nasıl: Tail sürekliliği (2.4) kurulduktan sonra: bakım gerektiren kuyruğun günün son varışına bir üst sınır koymak.

$$\theta_\text{son_varış}(\text{kuyruk t, gün h}) \;\le\; \text{bakım_başlangıcı}(t, h) - \text{hazırlık_süresi}$$

Etki: Operasyonel geçerlilik · **Zorluk:** Orta (2.4'e bağımlı) · **Gereken veri:** Bakım planı — mevcut veride YOK

2.8 λ'nın uçuş bazlı genelleştirilmesi

Ne: Tek bir λ yerine uçuş tipine/menziline göre değişen ceza katsayısı.

Neden: Şu an λ = 0,3 tüm uçuşlar için aynı. Ama bir uzun-menzil uçuşunu kaydırmanın maliyeti (slot, ekip layover, bağlantı dalgası) bir kısa-menzil uçuşununkinden çok farklıdır.

Nasıl: λ'yı olay bazlı bir vektöre çevirmek. Formülasyon değişmez, sadece katsayı indekslenir — hâlâ tam doğrusal.

$$\text{ceza} \;=\; \Sigma_r \;\lambda_r \cdot \Sigma_t \,|t - \text{cur_r}| \cdot x_\{r,t\}$$

$$\lambda_r \;=\; \lambda_0 \cdot (1 + \kappa \cdot \text{blok_süresi_r} \,/\, \text{ortalama_blok}) \quad \text{örnek bir kural}$$

Etki: Kaydırmalar operasyonel maliyete göre dağılır · **Zorluk:** Düşük

2.9 Minimum hizmet garantisi kısıtı

Ne: Belirli stratejik pazarlarda en az n bağlantı sunulmasını zorlamak.

Neden: 28 pazar-günde hiç bağlantı sunulamıyor. Ayrıca B2 kapalı olduğu için model uygun bir bağlantıyı bilinçli olarak sunmayabiliyor (~%3,5).

Nasıl: Doğrudan doğrusal bir alt sınır.

$$\Sigma_\{\pi \in \Pi_od,h\}\; y_\pi \;\geq\; n_min(o,d) \qquad \forall\, (o,d) \in \text{stratejik pazarlar}$$

Etki: Ticari politikayı modele taşır · **Zorluk:** Çok düşük

2.10 Hub kapasitesini kapı / apron / bagaj boyutuna genişletmek

Ne: 10 dk kova sayımı yerine çok boyutlu kaynak kısıtı.

Neden: Şu an kapasite yalnız hareket sayısıyla ölçülüyor. Gerçekte geniş gövde uçak iki kapı yeri kaplar, apron ve bagaj sistemi ayrı darboğazlardır.

Nasıl: Her uçuşa bir kaynak tüketim katsayısı verip kova başına toplam tüketimi sınırlamak.

$$\Sigma_\{r\}\; c_r \cdot x_\{r,t\} \;\leq\; \text{Cap_kaynak,t} \qquad \forall\, t, \forall\, \text{kaynak}$$

$$c_r = \text{uçuş r'nin kaynak tüketimi (ör. geniş gövde 2, dar gövde 1)}$$

Etki: Kapasite modelinin gerçekçiliği · **Zorluk:** Düşük (kod) · **Gereken veri:** Uçak tipi ve kapı/apron kapasite matrisi

2.11 Çok aktarmalı (double-connection) bağlantılar

Ne: İki aktarmalı yolculukları da modele almak.

Neden: Şu an yalnız tek aktarmalı (A→IST→B) bağlantılar var. Bazı uzak pazarlar ancak iki aktarmayla erişilebilir ve rakipler bunları sunuyor olabilir.

Nasıl: Π kümesini üçlü bacak kombinasyonlarıyla genişletmek. Model boyutu belirgin büyür; ön-işleme (ulaşılamayan kombinasyonları eleme) kritik hâle gelir.

Etki: Pazar kapsamı artar · **Zorluk:** Orta–Yüksek (kombinatoryal patlama riski)

2.12 Frekans kararı — kaç sefer yapılacağı

Ne: Sadece saatleri değil, sefer sayısını da karar değişkeni yapmak.

Neden: Şu an uçuş seti sabit; sadece saatleri oynatıyoruz. Gerçek ağ planlamada «bu hatta 1 mi 2 mi sefer» de bir karardır.

Nasıl: Her (hat, gün) için bir frekans değişkeni ve uçak/kapasite kaynak kısıtları. Bu, problemi klasik «schedule design» problemine dönüştürür ve çok büyütür.

Etki: Çok yüksek — ama farklı bir problem sınıfı · **Zorluk:** Araştırma konusu · **Gereken veri:** Filo büyüklüğü, uçak tipi maliyetleri

3. Çözüm yöntemi iyileştirmeleri

Bu bölüm modeli değiştirmez, ONU NASIL ÇÖZDÜĞÜMÜZÜ değiştirir. Hedef: %1,18'lik gap'i kapatmak ve daha büyük örneklere ölçeklenmek.

3.1 Ticari çözücü + indicator constraints

Ne: Gurobi veya CPLEX'e geçip big-M'li koşulları indicator constraint olarak yazmak.

Neden: En büyük kazanç E2'de: indicator constraint «y=1 İSE bu kısıt geçerli» ifadesini BIG-M OLMADAN doğrudan destekler. Çözücü bunu içeride özel olarak işler ve LP relaxation bozulmaz. E2'yi kapatmamızın çözücü kaynaklı gerekçesi ortadan kalkar. Ayrıca güçlü presolve ve otomatik cut üretimi %1,18'lik gap'i belirgin daraltır.

Nasıl: Pyomo kullandığımız için model kodu DEĞİŞMEZ — sadece `CFG['solver']` değeri değişir. Indicator constraint için Pyomo'nun `Disjunct`/GDP eklentisi veya doğrudan gurobipy `addGenConstrIndicator` kullanılır.

```
# Big-M ile (mevcut):
m.cons.add(gap[pi] >= L - Mlo * (1 - m.y[pi]))

# Indicator constraint ile (Gurobi):
model.addGenConstrIndicator(y[pi], True, gap[pi] >= L)
#   -> M YOK, LP relaxation bozulmaz, E2 acik kalabilir
```

Etki: Gap'in belirgin daralması + E2'nin açılabilmesi · **Zorluk:** Düşük (teknik) — lisans meselesi · **Gereken veri:** Akademik veya ticari Gurobi/CPLEX lisansı

Önemli uyarı — bunu sunumda mutlaka ekleyin: Gurobi E2'nin ÇÖZÜCÜ kaynaklı gerekçesini ortadan kaldırır ama VERİ kaynaklı gerekçesini KALDIRMAZ. Aktif yön çiftlerinin %76'sı Γ=30 dk eşiğini 465 dakikaya varan farklarla ihlal ediyor. Yani en güçlü çözücüyle bile E2'yi SERT uygulamak ya infeasible verir ya kazancı çökertir. Doğru çözüm yine soft-E2'dir (§2.1) — Gurobi onu sadece çok daha kolay çözülebilir kılar.

3.2 Large Neighborhood Search (LNS) ile cilalama

Ne: Çözümün bir bölümünü sabitleyip kalanını yeniden optimize etmeyi tekrarlamak.

Neden: Aşama-2 tek bir uzun branch-and-bound koşusu yapıyor ve 1.800 saniyede %1,18'de kalıyor. LNS, aynı bütçeyi çok sayıda küçük ve KOLAY alt probleme bölerek genellikle daha derine iner.

Nasıl: Her turda uçuşların rastgele (veya akıllıca seçilmiş) %85'ini mevcut değerine sabitleyip kalan %15'i serbest bırakmak. Alt problem çok küçük olduğu için saniyeler içinde OPTİMAL çözülür. Kabul edilirse yeni incumbent olur.

```
for tur in range(N_TUR):
    serbest = rastgele_sec(olaylar, oran=0.15)     # ya da: en cok kisitlanan pazarlar
    for e in olaylar:
        if e not in serbest:
            m.x[e, t_mevcut[e]].fix(1)             # geri kalani DONDUR
    res = solve_model(m)                           # kucuk problem -> saniyeler
    if amac(m) > en_iyi: en_iyi = amac(m); kaydet()
    unfix_all(m)
```

Etki: Gap'in daralması — literatürde benzer problemlerde %30–60 gap azalması tipiktir · **Zorluk:** Düşük — mevcut altyapıyla ~100 satır

3.3 Fix-and-optimize (pazar veya coğrafya bazlı)

Ne: LNS'in yapılandırılmış hâli: alt problemleri rastgele değil anlamlı bloklarla seçmek.

Neden: Rastgele seçim bazen birbiriyle ilgisiz uçuşları serbest bırakır ve iyileşme üretmez. Aynı dalgada veya aynı coğrafyada uçan uçuşlar birlikte serbest bırakıldığında iyileşme olasılığı çok daha yüksektir.

Nasıl: Blokları şöyle tanımlamak: (a) aynı zaman dalgasındaki tüm uçuşlar, (b) aynı bölgeye uçan tüm uçuşlar, (c) en düşük ρ-katkı üreten pazarların uçuşları, (d) en çok kısıtlanan (bandı bırakılmış) uçuşlar.

Etki: LNS'ten daha hızlı yakınsama · **Zorluk:** Düşük

3.4 Daha uzun cilalama bütçesi + deterministik paralel

Ne: 1.800 saniye yerine 4–8 saat; mümkünse deterministik çok-çekirdek.

Neden: %1,18 gap kabaca 40.000 puanlık bir potansiyele karşılık geliyor — yani +%4,72 yerine en iyi ihtimalle ~+%5,9. Bunun bir kısmı sadece daha uzun çalışarak alınabilir.

Nasıl: threads=1 kısıtımız determinizm içindi. Gurobi'nin deterministik paralel modu (`Method`, `ConcurrentMIP` ayarları) hem paralellik hem tekrar-üretilebilirlik verir. HiGHS ile kalınacaksa: farklı tohumlarla birden çok tek-iş-parçacıklı koşu yapıp en iyisini almak (portfolio yaklaşımı) da determinizmi korur.

Etki: Gap'in doğrudan daralması · **Zorluk:** Çok düşük

3.5 Benders ayrıştırması

Ne: Problemi «zaman atama (master)» ve «bağlantı/ödül değerlendirme (subproblem)» olarak ayırmak.

Neden: Mevcut ayrıştırmamız GÜNE göre. Yapısal bir alternatif: x (saat ataması) master'da kalır, y/z/w/β/γ (ödül mekaniği) subproblem'e iner. Subproblem x verildiğinde çok kolaydır — hatta çoğu kısmı kapalı formda hesaplanabilir.

Nasıl: Master her turda bir x önerisi üretir; subproblem o x için gerçek ödülü hesaplar ve master'a bir kesme (Benders cut) döndürür. Ödül fonksiyonu x'te içbükey olmadığı için klasik Benders yerine «combinatorial Benders» veya «logic-based Benders» gerekir.

Etki: Çok büyük örneklerde (çok haftalık) ölçeklenebilirlik · **Zorluk:** Yüksek — araştırma düzeyi uygulama

3.6 Column generation (sütun üretimi)

Ne: Tüm aday bağlantıları baştan üretmek yerine, gerektikçe üretmek.

Neden: Şu an 7.577 aday bağlantının hepsi modelde. Çok aktarmalı bağlantılar (§2.11) eklenirse bu sayı patlar. Column generation, yalnız «kârlı» olma potansiyeli olan bağlantıları modele alır.

Nasıl: Ana problem yalnız mevcut bağlantı kümesiyle çözülür; fiyatlandırma alt problemi (pricing) «negatif indirgenmiş maliyetli» yeni bir bağlantı arar. Bulunamazsa optimal.

Etki: Çok aktarmalı senaryolarda ölçeklenebilirlik · **Zorluk:** Yüksek

3.7 Rolling horizon (kayan ufuk)

Ne: Sezonluk ölçeğe çıkarken haftaları kayan pencerelerle çözmek.

Neden: Bir sezon 26 haftadır. Tek MIP olarak çözülemez. Ama haftalar arası bağ zaten zayıftır (yalnız tarife düzenliliği).

Nasıl: 3 haftalık pencerelerle ilerlemek: hafta 1–3 çözülür, hafta 1 sabitlenir, hafta 2–4 çözülür… Her pencere mevcut iki aşamalı hattımızı kullanır.

Etki: Sezonluk ölçeğe geçiş · **Zorluk:** Orta

3.8 Kullanıcı kesmeleri (user cuts)

Ne: Probleme özgü geçerli eşitsizlikler ekleyerek LP relaxation'ı sıkmak.

Neden: Monoton slot kısıtları ($z_j \geq z_{j+1}$) zaten bir tür kesme. Daha fazlası mümkün: örneğin bir olayın kapasite dolu bir kovaya atanamayacağını ifade eden clique kesmeleri veya bir pazarda ulaşılabilir maksimum bağlantı sayısına dair üst sınırlar.

Nasıl: Ön-işlemede statik olarak veya çözüm sırasında callback ile eklenir.

Örnek — pazar-gün başına ulaşılabilir üst sınır:

$$\Sigma_{j}\ z_{od,h,j} \leq n_{max}(o,d,h) \qquad \text{(pencere analiziyle önceden hesaplanır)}$$

Etki: Gap'in daralması · **Zorluk:** Orta

3.9 Simetri kırma

Ne: Eşdeğer çözümleri eleyerek arama ağacını küçültmek.

Neden: λ cezası zaten büyük bir simetri kaynağını (bedava kaydırma) kırdı. Kalan simetri: aynı pazarda birbirinin yerine geçebilen bağlantılar.

Nasıl: Sözlüksel (lexicographic) sıralama kısıtları eklemek. Dikkat: yanlış uygulanırsa optimal çözümü keser.

Etki: Orta düzeyde hızlanma · **Zorluk:** Orta (dikkatli olunmalı)

3.10 Çözücü portföyü ve otomatik parametre ayarı

Ne: Farklı çözücü ayarlarını paralel deneyip en iyisini almak.

Neden: HiGHS'in `mip_heuristic_effort`, presolve seviyesi, dallanma kuralı gibi onlarca parametresi var. Biz yalnız birkaçını elle ayarladık.

Nasıl: SMAC / irace gibi otomatik algoritma konfigürasyon araçlarıyla ayar taraması; ya da basitçe 4–8 farklı ayarı paralel koşup en iyisini almak.

Etki: %10–30 hızlanma tipiktir · **Zorluk:** Düşük

4. Belirsizlik ve sağlamlık (stokastik / robust)

Mevcut modelimiz tamamen DETERMİNİSTİKTİR: her uçuş tam zamanında kalkar ve iner. Gerçekte gecikmeler vardır ve sıkı bağlantılar kaçırılır. Bu bölüm belirsizliği modele sokar.

4.1 Bağlantı güvenilirliği metriği (ölçüm önce)

Ne: Her bağlantı için «kaçırılma olasılığını» hesaplayıp raporlamak.

Neden: Modelimiz MCT sınırına doğru itiyor: tam 60 dakikada bekleyen bağlantı sayısı 202'den (%3,0) 541'e (%7,8) çıkıyor. Bu bağlantılar 15 dakikalık bir gecikmede ölür. Şu an bunu ne ölçüyoruz ne de amaçta dikkate alıyoruz.

Nasıl: Optimizasyon değiştirmeden, çözüm sonrası bir analiz katmanı: gelen uçuşun gecikme dağılımı verildiğinde P(bağlantı yakalanır) hesaplanır ve raporlanır. Bu, ilk adım olarak en ucuz ve en bilgilendirici olanıdır.

$$\text{P_yakala}(\pi) \;=\; \text{P}(\;\text{gecikme_gelen} \;\le\; \text{gap_}\pi - \text{MCT_gerçek}\;)$$

$$\text{Beklenen etkin bağlantı sayısı} \;=\; \Sigma_\pi \;\text{y_}\pi \cdot \text{P_yakala}(\pi)$$

Etki: Sağlamlığın ölçülebilir hâle gelmesi · **Zorluk:** Düşük · **Gereken veri:** Uçuş bazlı gecikme dağılımı (tarihsel OTP verisi)

4.2 Şans kısıtlı (chance-constrained) bağlantılar

Ne: «Her sunulan bağlantı en az %90 olasılıkla yakalanmalı» kuralı.

Neden: Deterministik [60,300] penceresi, gecikme riskini hiç görmüyor. Şans kısıtı bunu doğrudan modele sokar.

Nasıl: Gecikme dağılımı bilindiğinde, %90 güven için gereken minimum bekleme bir SABİT olarak hesaplanabilir. Yani şans kısıtı, uçuş bazlı DEĞİŞKEN bir alt sınıra dönüşür — model doğrusal kalır. Bu çok zarif bir dönüşümdür.

$$\begin{gathered}\text{P}(\;\text{gap_}\pi \ge \text{MCT} + \text{gecikme}\;) \;\ge\; 0{,}90 \\ \Downarrow \quad (\text{gecikme dağılımı biliniyorsa}) \\ \text{gap_}\pi \;\ge\; \text{MCT} + \text{F}^{-1}(0{,}90) \;=\; \text{L_}\pi \qquad \leftarrow \text{uçuşa özel SABİT alt sınır}\end{gathered}$$

Etki: Sağlam tarife; kazançta ölçülebilir bir düşüş · **Zorluk:** Orta · **Gereken veri:** Gecikme dağılımları

4.3 Kaçırılan bağlantı maliyetini amaca eklemek

Ne: Ödülü, beklenen yakalanma olasılığıyla ağırlıklandırmak.

Neden: Şans kısıtı sert bir eşik koyar. Alternatif, riski amaçta fiyatlamaktır — böylece model risk ile kazanç arasında kendi dengesini kurar.

Nasıl: P_yakala(π) her aday saat kombinasyonu için önceden hesaplanabilir bir SABİTTİR (çünkü bekleme o kombinasyonda bellidir). Dolayısıyla amaç doğrusal kalır — aynı §2.2'deki kalite ağırlığı numarası.

$$\begin{aligned}\text{AMAÇ} \;=\; \Sigma\,\rho \cdot [\;\Sigma\text{_j}\; W^{c}\text{_j} \cdot \text{P_yakala},\pi \cdot \text{z_j} \;+\; \text{sıralama ödülü}\;] \\ -\;\text{c_misconnect} \cdot \Sigma_\pi\, \text{y_}\pi \cdot (1 - \text{P_yakala},\pi)\end{aligned}$$

Etki: Risk-kazanç dengesi modele girer · **Zorluk:** Orta · **Gereken veri:** Gecikme dağılımları + kaçırılan bağlantı birim maliyeti

4.4 Senaryo bazlı iki aşamalı stokastik programlama

Ne: Birden çok gecikme senaryosu üretip beklenen faydayı en büyüklemek.

Neden: Tek bir olasılık dağılımı yerine, korelasyonlu gecikmeleri (hava durumu, ATC tıkanıklığı) yakalayan senaryolar kullanmak daha gerçekçidir.

Nasıl: Birinci aşama: tarife kararı (x) — tüm senaryolarda ortak. İkinci aşama: her senaryoda hangi bağlantıların gerçekleştiği. Amaç: birinci aşama + senaryolar üzerinden beklenen ikinci aşama faydası.

$$\max\; \Sigma\text{_s}\;\; \text{p_s} \cdot \text{Fayda_s}(\text{x}, \text{y_s})$$

$$\text{x tüm senaryolarda aynı (here-and-now), y_s senaryoya özel (wait-and-see)}$$

Etki: En gerçekçi sağlamlık modeli · **Zorluk:** Yüksek — model senaryo sayısıyla çarpan büyür (Benders ile birlikte)

4.5 Robust optimizasyon (belirsizlik kümesi)

Ne: Olasılık dağılımı yerine «en kötü durum» garantisi.

Neden: Gecikme dağılımı güvenilir değilse (veya yoksa), robust yaklaşım «gecikme şu aralıkta olabilir, en kötü durumda bile şu performansı garanti ederim» der.

Nasıl: Bütçeli belirsizlik (Bertsimas–Sim) yaklaşımı: aynı anda en fazla Γ tane uçuşun gecikeceğini varsaymak. Bu, doğrusal bir robust karşılık üretir.

Etki: Dağılım verisi olmadan sağlamlık · **Zorluk:** Yüksek

4.6 Tarihsel gecikme verisiyle kalibrasyon

Ne: Modelin varsayımlarını gerçek performans verisiyle doğrulamak.

Neden: τ = 45 dakika ve MCT = 60 dakika sabitleri şartnameden geliyor. Gerçekte MCT terminale, iç/dış hat kombinasyonuna ve bagaj transferine göre değişir.

Nasıl: Tarihsel OTP (on-time performance) ve bağlantı verisiyle, terminal ve hat tipine göre gerçekçi MCT dağılımları çıkarmak; ardından L'yi tek sabit yerine (o,d,terminal) bazlı bir matrise çevirmek.

Etki: Modelin gerçekliğe kalibrasyonu · **Zorluk:** Orta · **Gereken veri:** Tarihsel OTP ve bağlantı gerçekleşme verisi

5. Rekabet dinamikleri — rakip artık sabit değil

Mevcut modelde rakip tarifeleri SABİT veridir. Ölçtük: galibiyetlerimizin medyan marjı 110 dakika, ama %15,4'ü 30 dakikadan az marjla kazanılıyor. Rakipler tepki verirse bu kısım risk altına girer.

5.1 En iyi tepki (best response) iterasyonu

Ne: Rakiplerin de optimize ettiğini varsayıp sırayla çözmek.

Neden: «Rakip sabit» varsayımı tüm rekabet analizinin temel zayıflığıdır. Bir hakem haklı olarak «Emirates 20 dakika oynatırsa ne olur?» diye sorabilir.

Nasıl: THY optimize eder → rakip kendi tarifesini bizim çözümümüze karşı optimize eder → THY tekrar optimize eder… Bu iterasyon bir Nash dengesine yakınsayabilir (garanti yok, ama pratikte genelde yakınsar).

Etki: «Rakip sabit» itirazına doğrudan cevap; kazancın ne kadarının kalıcı olduğunu gösterir · **Zorluk:** Yüksek — rakip için de bir model kurmak gerekir · **Gereken veri:** Rakip hub yapısı ve uçuş setleri

5.2 Stackelberg (lider–takipçi) modeli

Ne: THY önce hamle yapar, rakip ona en iyi tepkiyi verir; THY bunu ÖNGÖREREK seçer.

Neden: Best response iterasyonundan daha güçlüdür: THY, rakibin tepkisini hesaba katarak kararını verir. Sonuç «tepkiye dayanıklı» bir tarifedir.

Nasıl: İki seviyeli (bilevel) programlama. Alt seviye rakibin problemi, üst seviye bizimki. Bilevel MIP çok zordur; genelde KKT koşullarıyla tek seviyeye indirgenir veya kesme tabanlı yöntemler kullanılır.

Etki: Teorik olarak en doğru rekabet modeli · **Zorluk:** Araştırma konusu

5.3 Marj tabanlı sağlamlık kısıtı (pratik ara çözüm)

Ne: «Her galibiyet en az X dakika marjla kazanılsın» kuralı.

Neden: 5.1 ve 5.2 ağır. Pratik ve ucuz bir ara çözüm: rakibi kıl payı geçmek yerine güvenli marjla geçmeyi zorlamak.

Nasıl: Yenme koşulundaki eşiği bir güvenlik payıyla sıkılaştırmak — tek satırlık değişiklik.

$$\text{Mevcut:}\quad J_{\pi} \le T_{comp,k}$$
$$\text{Sağlam:}\quad J_{\pi} \le T_{comp,k} - M_{güvenlik} \qquad (\text{ör. } M = 20 \text{ dakika})$$

Etki: Ölçülebilir sağlamlık artışı; kazançta düşüş. Mevcut çözümde <20 dk marjlı 809 galibiyet var (%11,2) — bunlar risk altında · **Zorluk:** Çok düşük

5.4 Rakip tepki senaryolarıyla duyarlılık

Ne: «Rakipler tarifelerini ±X dakika oynatırsa kazancımız ne olur?» analizi.

Neden: Bu, tam bir oyun modeli kurmadan rekabet riskini ölçmenin en ucuz yoludur.

Nasıl: T_comp değerlerini −10, −20, −30 dakika kaydırıp çözümü YENİDEN DEĞERLENDİRMEK (yeniden optimize etmeden). Kaç galibiyetin kaybedildiği doğrudan sayılır.

Etki: Rekabet riskinin sayısallaşması — ölçtük: 30 dk kaymada galibiyetlerin %15,4'ü risk altında · **Zorluk:** Çok düşük

6. Talep ve gelir modellemesi

Şu an ρ maskelenmiş bir önem endeksi (1–1000) ve amaç fonksiyonu «bağlantı sayısı + sıralama» üzerinden ilerliyor. Gerçek gelir modeliyle bağlantı kurulduğunda çözüm doğrudan iş değerine çevrilebilir.

6.1 ρ yerine gerçek O–D gelir verisi

Ne: Maskelenmiş endeksi gerçek gelir/yolcu sayısıyla değiştirmek.

Neden: «+153.512 puan kaç TL?» sorusunun cevabı şu an yok — ve doğru cevap «veremeyiz», çünkü ρ maskeli. Gerçek veriyle bu doğrudan TL cinsinden ifade edilebilir.

Nasıl: Model yapısı HİÇ DEĞİŞMEZ; sadece amaç katsayıları değişir. Bu, mimarinin en güçlü yanlarından biri: iş değeri katmanı takılabilir.

Etki: Sonucun doğrudan TL cinsinden ifade edilmesi · **Zorluk:** Çok düşük (kod) · **Gereken veri:** O–D bazlı gerçek gelir ve yolcu sayısı

6.2 QSI / logit tabanlı pazar payı modeli

Ne: «Kaç bağlantı sunduk» yerine «kaç yolcu kazandık» hesaplamak.

Neden: Mevcut ödül yapısı (azalan getiri + sıralama) şartnameden geliyor ve bir vekildir. Havacılıkta standart yaklaşım QSI'dir (Quality of Service Index): her seçeneğe bir çekicilik skoru verilir ve pazar payı bu skorların oranıyla hesaplanır.

Nasıl: Bu DOĞRUSAL DEĞİLDİR (paylar oran içerir). İki yol var: (a) parçalı doğrusal yaklaşıklama, (b) dış döngüde iteratif çözüm — MIP çözülür, paylar hesaplanır, katsayılar güncellenir, tekrar çözülür.

$$\text{QSI_}\pi \;=\; \exp(\,\beta_1 \cdot \text{süre_}\pi + \beta_2 \cdot \text{aktarma_sayısı} + \beta_3 \cdot \text{kalkış_saati_uygunluğu}\,)$$

$$\text{pazar_payı_}\pi \;=\; \text{QSI_}\pi \;/\; \Sigma_\{\text{tüm seçenekler}\}\ \text{QSI}$$

Etki: Gerçek yolcu davranışına çok daha yakın bir amaç · **Zorluk:** Orta–Yüksek (doğrusal değil) · **Gereken veri:** Tarihsel pazar payı verisi (β katsayılarını kalibre etmek için)

6.3 Spill & recapture (taşma ve geri kazanım)

Ne: Koltuk dolduğunda yolcunun nereye gittiğini modellemek.

Neden: Şu an koltuk kapasitesi hiç yok — bir bağlantı ya sunuluyor ya sunulmuyor. Gerçekte uçak dolarsa yolcu «taşar» (spill) ve bir kısmı başka bir THY bağlantısına «geri kazanılır» (recapture), kalanı rakibe gider.

Nasıl: Bağlantı bazlı yolcu akış değişkenleri ve uçuş bazlı koltuk kapasitesi kısıtları eklemek. Bu, problemi bir ağ akışı problemiyle birleştirir.

Etki: Gelir tahmininin gerçekçiliği · **Zorluk:** Yüksek · **Gereken veri:** Uçak tipi, koltuk kapasitesi, tarihsel doluluk

6.4 Fiyat esnekliği ve gelir yönetimi bağlantısı

Ne: Süre kısaldığında fiyat priminin de artabileceğini modellemek.

Neden: Daha kısa yolculuk süresi sadece daha çok yolcu değil, daha yüksek fiyat da getirebilir (iş yolcusu süreye duyarlıdır).

Nasıl: Gelir katkısını süre kısalmasının doğrusal bir fonksiyonu olarak amaca eklemek.

Etki: İş değerinin daha doğru tahmini · **Zorluk:** Orta · **Gereken veri:** Fiyat–talep esneklik tahminleri

6.5 Yolcu segmentasyonu

Ne: İş / tatil / VFR yolcularının farklı tercihlerini ayrı modellemek.

Neden: İş yolcusu süreye ve kalkış saatine, tatil yolcusu fiyata duyarlıdır. Tek bir ρ ile hepsi aynı sayılıyor.

Nasıl: Segment bazlı ρ ve segment bazlı ödül fonksiyonu; toplam amaç segmentlerin ağırlıklı toplamı.

Etki: Daha isabetli pazar önceliklendirme · **Zorluk:** Orta · **Gereken veri:** Segment kırılımlı talep verisi

6.6 Kod paylaşımı (codeshare) ve ittifak bağlantıları

Ne: Star Alliance ortaklarının bağlantılarını da ürün olarak saymak.

Neden: Veride yalnız TK uçuşları «bizim» sayılıyor. Gerçekte codeshare bağlantıları da THY bileti olarak satılır ve gelir üretir.

Nasıl: Aday bağlantı kümesini ortak taşıyıcı bacaklarıyla genişletmek; ancak ortak uçuşların saatleri KARAR DEĞİLDİR (sabit girdi olarak girer).

Etki: Ürün kapsamının genişlemesi · **Zorluk:** Düşük (model) · **Gereken veri:** Codeshare anlaşma listesi

7. Yapay zekâ entegrasyonu

Bu bölüm neden özellikle önemli? Yarışmanın adı «YAPAY ZEKÂ DESTEKLİ Havayolu Optimizasyonu». Mevcut çözümümüz saf matematiksel optimizasyondur ve bu güçlü bir yaklaşımdır — ama jüri «yapay zekâ nerede?» diye sorabilir. Bu bölüm hem o soruya hazır cevap hem de gerçekten değerli bir genişleme yolu sunar.

Önemli bir çerçeveleme notu: matematiksel optimizasyon (özellikle MIP) yapay zekânın klasik ve en güvenilir dallarından biridir — «arama ve kısıt sağlama» geleneği. Makine öğrenmesi ise onun tamamlayıcısıdır, alternatifi değil. Aşağıdaki maddeler ikisini birleştirir.

7.1 ML ile bağlantı çekiciliği tahmini

Ne: Bir bağlantının gerçekten satılıp satılmayacağını öğrenen bir model kurmak.

Neden: Şu an bir bağlantının değeri sadece «pencerede mi» ve «rakibi geçiyor mu» ile belirleniyor. Gerçekte kalkış saati, gün, mevsim, terminal, yolcu segmenti gibi onlarca etken var.

Nasıl: Tarihsel rezervasyon verisiyle bir gradient boosting / lojistik regresyon modeli eğitip her aday bağlantı için bir «çekicilik skoru» üretmek. Bu skor, amaç fonksiyonunda §2.2'deki kalite çarpanının yerine geçer. KRİTİK: skor bir SABİT olarak girdiği için model DOĞRUSAL kalır — ML tahmin eder, MIP karar verir.

$$\text{ML modeli:}\quad \hat{q}_{\pi} = f(\text{gap}, \text{kalkış_saati}, \text{gün}, \text{mevsim}, \text{O–D özellikleri}, \dots)$$

$$\text{MIP amacı:}\quad \Sigma\, \rho \cdot \Sigma_j\, W^{c}_j \cdot \hat{q}_{\pi} \cdot z_j \quad \leftarrow \hat{q} \text{ SABİT, doğrusallık korunur}$$

Etki: Amaç fonksiyonunun gerçek talebe kalibrasyonu — en yüksek katma değerli ML uygulaması · **Zorluk:** Orta · **Gereken veri:** Tarihsel rezervasyon / arama verisi

7.2 Gecikme tahmin modeli → dinamik MCT

Ne: Uçuş bazlı gecikme olasılığını öğrenip bağlantı alt sınırını kişiselleştirmek.

Neden: MCT şu an tek bir sabit (60 dakika). Gerçekte Frankfurt'tan gelen bir uçuşun gecikme riski ile Ankara'dan gelenin riski çok farklıdır.

Nasıl: Hava durumu, tarihsel OTP, sezon, saat gibi özniteliklerle bir regresyon modeli eğitmek; çıktısını §4.2'deki şans kısıtına besleyerek uçuşa özel L_{π} üretmek.

$$\text{ML:}\quad \hat{g}\text{ecikme_dağılımı}(r) = f(\text{istasyon}, \text{saat}, \text{mevsim}, \text{tarihsel OTP}, \dots)$$
$$\text{MIP:}\quad \text{gap}_{\pi} \geq \text{MCT} + Q_{90}(\hat{g}\text{ecikme}(r_1)) \quad \leftarrow \text{uçuşa özel SABİT alt sınır}$$

Etki: Sağlamlık + gereksiz muhafazakârlıktan kaçınma · **Zorluk:** Orta · **Gereken veri:** Tarihsel OTP verisi

7.3 ML destekli dallanma (learning to branch)

Ne: Çözücünün dallanma kararlarını öğrenilmiş bir politikayla yönlendirmek.

Neden: Branch and bound'da hangi değişken üzerinde dallanılacağı çözüm süresini dramatik etkiler. Varsayılan kurallar (pseudocost, strong branching) geneldir; bu probleme özel bir politika daha iyi olabilir.

Nasıl: Strong branching kararlarını «öğretmen» olarak kullanıp bir sınıflandırıcı eğitmek (imitation learning). Literatürde bu yaklaşım MIP çözüm sürelerinde %20–50 iyileşme raporlamıştır.

Etki: Çözüm süresinde belirgin azalma · **Zorluk:** Yüksek — çözücü callback'i gerekir

7.4 Predict-then-optimize: ML ile başlangıç çözümü

Ne: Geçmiş çözümlerden öğrenip iyi bir MIP-start üretmek.

Neden: Aşama-2 şu an aşama-1'in çözümünden başlıyor. Bir ML modeli, benzer örneklerdeki çözümlerden öğrenerek daha iyi bir başlangıç noktası önerebilir.

Nasıl: Her uçuş için «bu uçuş kaç dakika kayacak» tahmini yapan bir regresyon modeli; tahmin edilen tarife bir onarım adımından geçirilip MIP-start olarak verilir.

Etki: Daha iyi başlangıç → daha küçük gap · **Zorluk:** Orta · **Gereken veri:** Farklı senaryolarda üretilmiş çözüm arşivi

7.5 Sürrogat (vekil) model ile hızlı senaryo taraması

Ne: MIP'i çalıştırmadan sonucu tahmin eden hızlı bir model.

Neden: Duyarlılık analizi her parametre için tam bir MIP koşusu gerektiriyor (9 koşu × 7 gün × 150 s). Yüzlerce senaryo denemek imkânsız.

Nasıl: Birkaç yüz MIP koşusundan (parametre → sonuç) bir sürrogat model eğitmek. Sonra parametre uzayını saniyeler içinde taramak ve yalnız umut vaat eden noktalarda gerçek MIP çalıştırmak (Bayesian optimization).

Etki: Parametre uzayının çok daha geniş taranması · **Zorluk:** Orta

7.6 Pekiştirmeli öğrenme ile kayan ufuk kararları

Ne: Sezonluk planlamada hangi haftanın ne zaman yeniden optimize edileceğini öğrenmek.

Neden: Rolling horizon (§3.7) uygulanırsa, «ne zaman yeniden çöz» bir karar hâline gelir.

Nasıl: Durum = mevcut tarife + talep sinyalleri; eylem = yeniden optimize et / bekle; ödül = kazanç − değişim maliyeti. Bu bir Markov karar süreci olarak modellenebilir.

Etki: Dinamik planlama · **Zorluk:** Araştırma konusu

7.7 Kümeleme ile pazar segmentasyonu

Ne: Benzer davranışlı pazarları otomatik gruplayıp parametreleri grup bazlı ayarlamak.

Neden: λ, L, kalite ağırlıkları gibi parametreler şu an tüm pazarlar için aynı. Ama kısa menzil yoğun pazarlarla uzun menzil seyrek pazarlar çok farklı davranıyor.

Nasıl: Pazarları (ρ, N_rakip, aday sayısı, ortalama J, mevsimsellik) uzayında kümelemek ve her küme için ayrı parametre seti kalibre etmek.

Etki: Parametre ayarının incelmesi · **Zorluk:** Düşük

7.8 Anomali tespiti ile veri kalitesi kontrolü

Ne: Veri sağlık kontrolünü kural tabanlıdan öğrenme tabanlıya çevirmek.

Neden: Mevcut 01_data_health_check.py elle yazılmış kurallarla çalışıyor ve beş problem buldu. Ama bilmediğimiz problemleri bulamaz.

Nasıl: Izolation Forest / autoencoder gibi denetimsiz yöntemlerle beklenmedik satırları işaretlemek. Gizli test verisinde bilmediğimiz anomaliler varsa yakalar.

Etki: Gizli veri güvenliği · **Zorluk:** Düşük

7.9 Doğal dil arayüzü ile senaryo sorgulama

Ne: «VCE pazarını koruyarak yeniden optimize et» gibi komutları modele çevirmek.

Neden: Planlamacı MIP bilmez. Doğal dille senaryo tanımlayabilmesi aracın benimsenmesini kökten değiştirir.

Nasıl: Bir LLM katmanı doğal dil isteğini CFG değişikliklerine ve ek kısıtlara çevirir; optimizasyon yine deterministik MIP olarak çalışır. LLM karar VERMEZ, sadece isteği FORMÜLE eder — bu ayrım kritiktir.

Etki: Kullanılabilirlik ve benimsenme · **Zorluk:** Orta

8. Yazılım mimarisi ve ürünleşme

Mevcut kod bir yarışma teslimi olarak çok iyi durumda: modüler, tek komutla çalışıyor, deterministik. Ama bir ÜRÜN değil. Bu bölüm o yolu anlatıyor.

8.1 Konfigürasyonu koddan ayırmak (YAML / TOML)

Ne: CFG sözlüklerini harici bir yapılandırma dosyasına taşımak.

Neden: Şu an CFG Python kodunun içinde. Bir parametre değiştirmek için kaynak kodu düzenlemek gerekiyor — bu, kod bilmeyen bir planlamacı için imkânsız ve senaryo yönetimini zorlaştırıyor.

Nasıl: `config/default.yaml` + senaryo bazlı override dosyaları. Kod dosyayı okur, komut satırı argümanları üzerine yazar.

```
# config/default.yaml
zaman:   {delta_min: 10, wdev_min: 60}
baglanti: {L_min: 60, U_min: 300}
kisitlar: {E2_balance: false, B2_force_offer: false, F_capacity: true}
ceza:    {lambda: 0.3}
cozucu:  {name: appsi_highs, threads: 1, seed: 0, mip_gap: 0.001}

# calistirma
python run_all.py --config config/senaryo_dar_pencere.yaml
```

Etki: Senaryo yönetimi ve kullanılabilirlik · **Zorluk:** Düşük

8.2 İki CFG'yi tek kaynakta birleştirmek

Ne: 02_data_prep ve 03_model'deki tekrarlanan sabitleri ortak bir modüle almak.

Neden: L_min=60, U_min=300, tau_min=45 üç ayrı dosyada bağımsız tanımlı (01, 02, 03). Biri değişirse diğerleri sessizce eski değerle çalışmaya devam eder ve rapor ile model birbirinden ayrışır.

Nasıl: `src/common/config.py` tek kaynak; diğerleri buradan import eder.

Etki: Sapma riskinin ortadan kalkması · **Zorluk:** Çok düşük

8.3 Docker ile ortam paketleme

Ne: Tüm bağımlılıkları içeren bir konteyner imajı.

Neden: «Doğrulanmış ortam: Python 3.13» diyoruz ama jürinin makinesinde 3.11 olabilir. Docker bu belirsizliği tamamen ortadan kaldırır.

Nasıl: Basit bir Dockerfile: python:3.13-slim tabanlı, requirements.lock kurulu, `docker run buke-thy python run_all.py`.

Etki: Tekrar-üretilebilirliğin garantiye alınması · **Zorluk:** Çok düşük

8.4 REST API servisi

Ne: Modeli bir HTTP servisi olarak sunmak.

Neden: Şu an her şey komut satırından çalışıyor. Bir planlama sistemi entegrasyonu için programatik arayüz gerekir.

Nasıl: FastAPI ile: `POST /optimize` (senaryo parametreleri) → iş kimliği; `GET /status/{id}` → ilerleme; `GET /result/{id}` → çözüm. Uzun süren çözümler için arka plan kuyruğu (Celery/RQ).

Etki: Sistem entegrasyonu · **Zorluk:** Orta

8.5 Senaryo yönetimi ve versiyonlama

Ne: Farklı parametre setleriyle üretilmiş çözümleri saklayıp karşılaştırmak.

Neden: Şu an her koşu `output/` klasörünü ÜZERİNE YAZIYOR. Bir önceki senaryoyu karşılaştırmak için elle yedek almak gerekiyor.

Nasıl: Her koşuya bir kimlik (parametre hash'i + zaman damgası) verip `runs/{id}/` altında saklamak; bir `compare` komutuyla iki koşuyu yan yana raporlamak.

Etki: Karar destek kullanımı için şart · **Zorluk:** Düşük

8.6 Yapılandırılmış loglama ve izlenebilirlik

Ne: print() yerine yapılandırılmış log kayıtları.

Neden: Şu an tüm çıktı print ile konsola gidiyor. Uzun koşularda ne olduğunu sonradan analiz etmek zor; makine tarafından okunabilir değil.

Nasıl: Python `logging` + JSON formatter. Her gün için çözüm süresi, gap, kısıt sayıları yapılandırılmış kayıt olarak tutulur ve sonradan analiz edilebilir.

Etki: Operasyonel izlenebilirlik · **Zorluk:** Düşük

8.7 Paralel gün çözümü

Ne: Güne göre ayrıştırmada günleri paralel çözmek.

Neden: 7 gün şu an sırayla çözülüyor (~1,5 dakika). Ama G bandı zinciri sıralı bağımlılık yaratıyor — bu yüzden doğrudan paralelleştirilemez.

Nasıl: İki yol: (a) bant zincirini kaldırıp günleri bağımsız çözmek, sonra aşama-2'de düzenliliği tam modelde kurmak (zaten öyle yapılıyor) — bu paralelliği açar; (b) çok senaryolu koşularda (duyarlılık taraması) senaryoları paralelleştirmek — bu daha kolay ve daha faydalı.

Etki: Duyarlılık taramasında 5–9 kat hızlanma · **Zorluk:** Düşük

8.8 CI/CD boru hattı

Ne: Her kod değişikliğinde otomatik test ve küçük ölçekli doğrulama koşusu.

Neden: Şu an bir değişiklik sonucu bozarsa bunu ancak elle koşarak fark ederiz.

Nasıl: GitHub Actions: her push'ta birim testler + küçük alt-örnek üzerinde uçtan uca koşu + doğrulayıcının PASS vermesi şartı.

Etki: Regresyon güvenliği · **Zorluk:** Düşük

9. Test, doğrulama ve kalite güvencesi

Mevcut durum: Projede 470 satırlık güçlü bir bağımsız doğrulayıcı var ve bu büyük bir artı. Ama HİÇ OTOMATİK BİRİM TEST YOK. Doğrulayıcı çözümün doğruluğunu kontrol ediyor; fonksiyonların doğruluğunu kontrol eden bir katman yok.

9.1 Birim testler (pytest)

Ne: Her yardımcı fonksiyon için otomatik test.

Neden: `dur_min()` altı farklı süre formatını işliyor; `snap()` yuvarlama yapıyor; `rank_reward()` lookup okuyor. Bunların hiçbiri test edilmiyor. Bir refaktör sırasında bozulursa fark edilmez.

Nasıl: En kritik 10 fonksiyon için test yazmak: dur_min (6 format), snap (kenar durumlar), rank_reward (eksik anahtar), rotation_pairs (yarım çiftler), current_baseline (bilinen küçük örnek), hafta normalizasyonu (7'nin katı olmayan fark).

```
def test_dur_min_tum_formatlar():
    assert dur_min(dt.time(2, 35)) == 155
    assert dur_min(dt.datetime(1899, 12, 30, 2, 35)) == 155
    assert dur_min('30.12.1899 02:35:00') == 155
    assert dur_min(pd.Timedelta(minutes=155)) == 155
    assert dur_min(155 / 1440.0) == 155
    assert np.isnan(dur_min(None))

def test_hafta_normalizasyonu_7nin_kati_degilse_dokunmaz():
    df = ornek_veri(fark_gun=3)          # 3 gun -> 7'nin kati DEGIL
    out = normalize_hafta(df)
    assert (out.ist_dep == df.ist_dep).all()
```

Etki: Regresyon güvenliği; refaktör cesareti · **Zorluk:** Düşük

9.2 Bilinen küçük örnek üzerinde uçtan uca regresyon testi

Ne: Elle doğrulanmış küçük bir örnekte beklenen sonucu sabitlemek.

Neden: Küçük alt-örnek zaten var ve amaç değeri biliniyor (14.384,38). Bu bir regresyon testi olarak otomatikleştirilebilir.

Nasıl: `test_kucuk_ornek_amaci_degismedi()` — model değişirse test kırılır ve neden değiştiğini açıklamak zorunda kalırsınız. Bu, sessiz sapmaları engeller.

Etki: Model davranışının kilitlenmesi · **Zorluk:** Çok düşük

9.3 IIS (infeasible alt sistem) analizi

Ne: Model çözümsüz olduğunda hangi kısıtların çeliştiğini otomatik bulmak.

Neden: B2 açıldığında model infeasible oluyor ve biz bunun sebebini ELLE (bisection ile) bulduk. IIS bunu otomatik yapar.

Nasıl: Gurobi `computeIIS()` doğrudan destekler. HiGHS'te yok; alternatif olarak kısıtları aileye göre isimlendirip (§1.13) deletion filtering uygulamak.

Etki: Hata ayıklama hızı · **Zorluk:** Düşük (Gurobi ile) / Orta (elle)

9.4 Property-based (özellik tabanlı) test

Ne: Rastgele üretilmiş girdilerle değişmezleri sınamak.

Neden: Elle yazılan testler düşündüğünüz durumları kontrol eder. Property-based test düşünmediğiniz durumları bulur.

Nasıl: Hypothesis kütüphanesi ile: «her rastgele geçerli girdide, çözümün toplam kaydırması ≤ Wdev × olay sayısı olmalı», «baseline ≤ optimize olmalı», «hiçbir olay iki saate atanamaz» gibi değişmezleri sınamak.

Etki: Beklenmedik kenar durumların bulunması · **Zorluk:** Orta

9.5 Doğrulayıcının konvansiyon bağımsızlığını artırmak

Ne: 05'in b_od ve beaten değerlerini model çıktısından değil, kendi başına hesaplaması.

Neden: Doğrulayıcı UYGULAMA hatalarına karşı bağımsız ama KONVANSİYON hatalarına karşı değil: sıralama ödülünü kontrol ederken b_od'yi model çıktı dosyasından okuyor (satır 346). Yani b_od yanlışsa yakalayamaz.

Nasıl: b_od'yi doğrulayıcının kendi kodunda, veri katmanından yeniden türetmek ve model çıktısıyla karşılaştırmak. Bu, doğrulayıcının kapsamını gerçekten genişletir.

Etki: «Bağımsız doğrulayıcı» iddiasının güçlenmesi · **Zorluk:** Düşük

9.6 Çapraz doğrulama: farklı veri dilimleriyle

Ne: Modeli verinin farklı alt kümelerinde çalıştırıp tutarlılığı sınamak.

Neden: Şu an tek bir hafta üzerinde çalışıyoruz. Modelin davranışının veriye özel olup olmadığını bilmiyoruz.

Nasıl: Günleri farklı kombinasyonlarda gruplayıp (ör. yalnız hafta içi, yalnız hafta sonu) koşmak ve iyileşme oranlarının benzer çıkıp çıkmadığına bakmak. Zaten gün bazlı iyileşmelerimiz dengeli (+19.111 … +23.677) — bu iyi bir işaret.

Etki: Genellenebilirlik kanıtı · **Zorluk:** Düşük

10. Görselleştirme ve karar destek arayüzü

Şu an dört statik PNG figür var. Bir planlamacının bu çözümü gerçekten kullanabilmesi için etkileşimli bir katman gerekir.

10.1 Hub dalga (wave) görselleştirmesi

Ne: IST'teki varış ve kalkış yoğunluğunu zaman ekseninde, öncesi/sonrası olarak göstermek.

Neden: Hub'ın kalp atışı olan dalga yapısı, tarifenin en sezgisel görselidir. Değişimin nereye dokunduğunu tek bakışta gösterir.

Nasıl: 10 dakikalık kovalarda varış/kalkış sayısı; mevcut ve optimize edilmiş iki seri üst üste. Kapasite sınırı (10/15) yatay çizgi olarak. Bu grafik aynı zamanda «mevcut tarife kapasiteyi 96 kovada aşıyor, biz aşmıyoruz» bulgusunu görsel olarak kanıtlar.

Etki: Sunumda çok etkili bir görsel · **Zorluk:** Çok düşük

10.2 Bağlantı bandı (connection band) grafiği

Ne: Bekleme süresi dağılımının öncesi/sonrası karşılaştırması.

Neden: Ürün kalitesi eleştirisine (§2.2) doğrudan görsel cevap: 60–90 dk bandının 1.187'den 1.332'ye çıktığını gösterir.

Nasıl: Yığılmış çubuk grafik: bantlar (60–90, 90–120, …) × (baseline, optimize).

Etki: Ürün kalitesi tartışmasında kanıt · **Zorluk:** Çok düşük

10.3 Pazar bazlı detay panosu (drill-down)

Ne: Bir pazara tıklayınca o pazardaki tüm bağlantıları, rakipleri ve değişimi görmek.

Neden: 775 pazarın hangisinde ne olduğunu CSV'den okumak zor. Bir planlamacı «VCE–JED'de ne oldu?» diye sorar.

Nasıl: Streamlit veya Dash ile basit bir arayüz: pazar seç → sunulan bağlantılar, her birinin J süresi, yenilen rakipler, sıralama değişimi.

Etki: Aracın benimsenmesi · **Zorluk:** Düşük

10.4 Öncesi/sonrası tarife karşılaştırma tablosu

Ne: Hangi uçuşun ne kadar kaydığını filtrelenebilir bir tabloda göstermek.

Neden: `optimized_schedule.csv` 4.554 satır; Excel'de açıp incelemek yorucu. Ayrıca bu dosya yalnız ayarlanabilir uçuşları içeriyor — sabit uçuşlarla birlikte tam tarife görüntüsü yok.

Nasıl: Sabit uçuşları da içeren birleşik bir tablo; kaydırma miktarına, güne, uçuş numarasına, bölgeye göre filtreleme.

Etki: Operasyonel inceleme kolaylığı · **Zorluk:** Çok düşük

10.5 Duyarlılık kokpiti

Ne: λ, Wdev, τ gibi parametreleri kaydırarak sonucun nasıl değiştiğini canlı görmek.

Neden: Duyarlılık analizi şu an statik bir tablo. Bir planlamacı «λ'yı biraz artırsam?» diye merak eder.

Nasıl: Önceden hesaplanmış senaryo arşivi + sürrogat model (§7.5) üzerine kaydırıcılı bir arayüz. Gerçek MIP çalıştırmadan anlık tahmin.

Etki: Parametre sezgisinin kazanılması · **Zorluk:** Orta

10.6 Otomatik yönetici özeti üretimi

Ne: Her koşudan sonra tek sayfalık, grafikli bir özet PDF üretmek.

Neden: Şu an results_summary.md var ama düz metin. Yönetim sunumu için grafikli bir özet gerekir.

Nasıl: Mevcut 04_export_docs altyapısı + figürlerin gömülmesi. Ayrıca §1.10'daki raporlanmayan sayaçlar da buraya girer.

Etki: Kurumsal iletişim · **Zorluk:** Düşük

11. Yol haritası — faz planı

81 maddenin hepsi aynı anda yapılamaz. Aşağıda dört faz hâlinde bir plan var. Fazlar birbirinin üzerine kuruluyor.

FAZ 0 — Hemen (1–2 hafta): ölçülmüş açıkları kapat

Amaç: ön-denetimde bulunan gerçek açıkları gidermek ve savunulabilirliği maksimuma çıkarmak. Hepsi mevcut mimariyle yapılabilir, yeni veri gerektirmez.

Madde	İş	Beklenen sonuç
1.1	Rotasyon filtresini çift bazlı yap	A2 kapsamı %57 → ~%100
1.2	Izgara-tutarlı baseline'ı da raporla	+%4,03 rakamının resmîleşmesi
1.3	Kök LP bound'unu raporla	Formülasyon kalitesi sayısallaşır
1.4	ρ çakışma politikasını belgele	Sessiz veri seçimi biter
1.6	Yenilen rakip tutarsızlığını gider	Çıktılar arası tutarlılık
1.7	Sürümleri sabitle + Docker	Tekrar-üretilebilirlik garantisi
1.10	Raporlanmayan sayaçları bas	Şeffaflık
9.1	Birim testler (en kritik 10 fonksiyon)	Regresyon güvenliği
9.2	Küçük örnek regresyon testi	Model davranışı kilitlenir
10.1	Hub dalga görselleştirmesi	Sunum için güçlü görsel

FAZ 1 — Kısa vade (1–3 ay): modeli tamamla

Amaç: bilinen model eksiklerini kapatmak ve gap'i daraltmak.

Madde	İş	Beklenen sonuç
3.1	Ticari çözücü + indicator constraints	Gap daralır, E2 açılabilir hâle gelir
2.1	Soft-E2	Model kapsamının en büyük açığı kapanır
2.2	Bağlantı kalitesi ağırlıklandırması	Ürün karması iyileşir
2.3	Pazar bazlı koruma kısıtı	24 kötüleşen pazar kontrol altına alınır
3.2	LNS ile cilalama	%1,18 gap'in belirgin daralması
3.4	Daha uzun cilalama bütçesi	Ek kazanç
1.14	Duyarlılık analizini genişlet	Parametre uzayı belgelenir
8.1	YAML konfigürasyon	Senaryo yönetimi
9.5	Doğrulayıcının kapsamını genişlet	«Bağımsız» iddiası güçlenir

FAZ 2 — Orta vade (3–9 ay): gerçek dünyaya bağla

Amaç: operasyonel uygulanabilirlik ve iş değeri. Bu fazın çoğu YENİ VERİ gerektirir.

Madde	İş	Gereken veri
2.4	Uçak kuyruk (tail) sürekliliği	Kuyruk bazlı görev dizisi
2.5	Dış istasyon slot ve curfew	Havalimanı slot pencereleri
2.6	Ekip (FDP) kısıtları	Pairing yapısı ve limitler
6.1	ρ yerine gerçek gelir verisi	O–D bazlı gelir
4.1–4.2	Gecikme analizi + şans kısıtı	Tarihsel OTP
7.1	ML ile bağlantı çekiciliği	Tarihsel rezervasyon
7.2	Gecikme tahmin modeli	Tarihsel OTP
8.4–8.5	REST API + senaryo yönetimi	—

10.3	Karar destek arayüzü	—

FAZ 3 — Uzun vade (9 ay+): araştırma sınırı

Madde	İş	Neden zor
6.2	QSI / logit pazar payı modeli	Doğrusal değil; iteratif çözüm gerekir
6.3	Spill & recapture	Ağ akışı problemiyle birleşir
4.4	Senaryo bazlı stokastik programlama	Model senaryo sayısıyla çarpan büyür
5.1–5.2	Oyun teorik rekabet modeli	Bilevel programlama
2.12	Frekans kararı (schedule design)	Farklı bir problem sınıfı
3.5–3.6	Benders / column generation	Araştırma düzeyi uygulama
7.3	ML destekli dallanma	Çözücü callback'i ve eğitim altyapısı

11.1 Bağımlılık haritası

```
FAZ 0 ------------------------------------------------------------------
  1.1 rotasyon filtresi ----+
  1.2 izgara-tutarli base --+--> savunulabilir taban
  9.1 birim testler --------+

FAZ 1 ------------------------------------------------------------------
  3.1 ticari cozucu --------+--> 2.1 soft-E2  (cok daha kolay olur)
                            +--> 9.3 IIS analizi
  2.2 kalite agirligi ------+--> 7.1 ML cekicilik  (ayni yapiyi kullanir)
  3.2 LNS -------------------+--> gap daralmasi

FAZ 2 ------------------------------------------------------------------
  2.4 tail surekliligi -----+--> 2.7 bakim pencereleri
  4.1 gecikme analizi ------+--> 4.2 sans kisiti --> 4.3 misconnect maliyeti
                            +--> 7.2 ML gecikme tahmini
  6.1 gercek gelir ---------+--> 6.4 fiyat esnekligi

FAZ 3 ------------------------------------------------------------------
  6.2 QSI/logit ------------+--> 6.3 spill & recapture
  4.2 sans kisiti ----------+--> 4.4 senaryo bazli stokastik
  5.1 best response --------+--> 5.2 Stackelberg
```

12. Sunumda hangileri söylenmeli?

Altın kural: «Bundan sonra ne yapardınız?» sorusuna 81 madde saymayın. ÜÇ madde söyleyin ve her biri için NEDEN sorusuna hazır olun. Üçü de mevcut bir zayıflığı kapatmalı — böylece «zayıflığımızı biliyoruz ve çözümü de biliyoruz» mesajı verilir.

12.1 Söylenecek üç madde

1. Soft-E2 (§2.1)

«E2'yi kapattık ve iki bağımsız kanıtla gerekçelendirdik. Ama doğru mühendislik hamlesi kısıtı düşürmek değil, ihlalini amaçta cezalandırmaktı. Formülasyonu hazır: her yön çiftine bir ihlal değişkeni ekleyip amaca ρ-ağırlıklı bir ceza terimi koyuyoruz. Böylece model E2'yi yasaklamıyor, minimize ediyor — ve infeasible olmuyor. Yapmama sebebimiz zaman bütçesiydi: μ katsayısı için λ'daki gibi bir Pareto taraması gerekiyor.»

2. Rotasyon kapsamının tamamlanması (§1.1)

«Ön-denetimimizde bulduk: rotasyon eşlerini uçuş bazlı filtrelediğimiz için 512 çiftin yalnız 291'i kullanılabilir hâle geliyor ve 221 uçuşta A2 kısıtı hiç kurulamıyor — bunların 100'ü çözümde oynatılmış. Düzeltmesi tek satır: filtreyi çift bazlı yapmak. Kök nedeni de bulduk: Flight Pairs dosyası W26 sezonundan, veri ise Ağustos 2025'ten.»

3. Bağlantı kalitesi ağırlıklandırması (§2.2)

«Amaç fonksiyonu 90 dakikalık bir bağlantıyla 300 dakikalığı aynı sayıyor — bu şartnamenin tanımı. Bir kalite çarpanı eklemek istiyoruz: 60–120 dakika tam puan, 240–300 dakika yarım puan. Kritik nokta şu: bekleme süresi her aday saat kombinasyonunda önceden bilindiği için bu çarpan bir SABİTTİR — yani model doğrusal kalıyor, hiçbir ek karmaşıklık gelmiyor.»

12.2 «Yapay zekâ nerede?» sorusu gelirse

Hazır cevap: «İki katmanlı cevabım var. Birincisi: matematiksel optimizasyon, özellikle karışık tamsayılı programlama, yapay zekânın klasik ve en güvenilir dallarından biridir — arama, kısıt sağlama ve optimalite kanıtı geleneği. Sezgisel bir yöntemin veremeyeceği bir şeyi veriyoruz: çözümümüzün optimumdan en fazla %1,18 uzakta olduğunun matematiksel kanıtı. İkincisi: makine öğrenmesi bunun alternatifi değil, tamamlayıcısıdır ve nereye takılacağını tam olarak biliyoruz. En değerlisi bağlantı çekiciliği tahmini: tarihsel rezervasyon verisiyle bir model eğitip her bağlantıya bir çekicilik skoru üretiriz ve bunu amaç fonksiyonuna katsayı olarak koyarız. Kritik nokta şu: skor bir sabit olarak girdiği için model doğrusal kalır — ML tahmin eder, MIP karar verir. İkincisi gecikme tahmini: uçuş bazlı gecikme dağılımını öğrenip minimum bağlantı süresini kişiselleştiririz. Üçüncüsü learning-to-branch: çözücünün dallanma kararlarını öğrenilmiş bir politikayla yönlendirmek — literatürde %20–50 hızlanma raporlanmış.»

12.3 «Bu gerçekten uygulanabilir mi?» sorusu gelirse

Hazır cevap: «Bugünkü hâliyle bir KARAR DESTEK aracı olarak uygulanabilir: planlamacıya «şu 1.067 hareketi ortalama 19 dakika oynatırsan şu kazancı alırsın» diyor ve her öneriyi gerekçesiyle birlikte CSV olarak veriyor. Üretim tarifesi olarak kullanılması için üç şey eksik ve üçünü de biliyoruz: uçak kuyruk sürekliliği, ekip görev süresi kısıtları ve dış istasyon slot pencereleri. Üçü de doğrusal kısıtlardır, model mimarisi değişmez — eksik olan veridir, yöntem değil. Gerçekçi bir tahmin: veri entegrasyonu için birkaç ay, ardından tarife planlama ekibiyle bir pilot sezon.»

12.4 Söylenmemesi gerekenler

- **Liste okumak:** 81 maddeyi saymak. Jüri detay değil, ÖNCELİK duymak ister.
- **Sınırsızlık:** «Her şeyi yapardık» demek. Kaynak kısıtı olmayan bir plan inandırıcı değildir.
- **Çözücüye sığınmak:** «Gurobi alsak her şey çözülürdü» demek. Doğru değil — E2'nin veri kaynaklı gerekçesi çözücüden bağımsızdır ve bunu bilmek sizi güçlü gösterir.

- **Abartmak:** Yapılmamış bir şeyi yapılmış gibi anlatmak. Bu dokümandaki her madde «yapılabilir» olarak yazılmıştır, «yapıldı» olarak değil.

12.5 Kapanış

> Bu yol haritasının asıl mesajı şudur: çözümümüzün sınırlarını biliyoruz, her sınır için somut bir çözüm yolumuz var ve bunların hangisinin ne kadar süreceğini de tahmin edebiliyoruz. Bir projeyi bitmiş göstermek kolaydır; nerede bitmediğini gösterebilmek ise onu gerçekten anlamış olmayı gerektirir.